~\Desktop\C++学习资料\Untitled-1.cpp

```cpp
#include <cuda_runtime.h>
#include <iostream>
#define TILE_WIDTH 16  // 块大小，通常设为16或32
__global__ void matrixMulTiled(const float *A, const float *B, float *C, int N) {
    // 这里的变量是所有线程共享的，生命周期仅在 Block 内
    __shared__ float ds_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_B[TILE_WIDTH][TILE_WIDTH];
    // 保存行列索引，简化后续代码
    int tx = threadIdx.x; int ty = threadIdx.y;
    int row = blockIdx.y * TILE_WIDTH + ty;
    int col = blockIdx.x * TILE_WIDTH + tx;
    float pValue = 0.0f;
    // 2. 核心循环：分阶段(Phase)遍历，每次处理一个 Tile
    // p 是阶段索引，总共有 N/TILE_WIDTH 个阶段
    for (int p = 0; p < (N + TILE_WIDTH - 1) / TILE_WIDTH; ++p) {
        // --- 步骤 A: 将数据从 Global Mem 加载到 Shared Mem ---
        // 加载 A 的 Tile 元素：注意列索引是 p * TILE_WIDTH + tx
        if (row < N && (p * TILE_WIDTH + tx) < N)
            ds_A[ty][tx] = A[row * N + (p * TILE_WIDTH + tx)];
        else
            ds_A[ty][tx] = 0.0f; // 边界填充0
        // 加载 B 的 Tile 元素：注意行索引是 p * TILE_WIDTH + ty
        if (col < N && (p * TILE_WIDTH + ty) < N)
            ds_B[ty][tx] = B[(p * TILE_WIDTH + ty) * N + col];
        else
            ds_B[ty][tx] = 0.0f;
        __syncthreads();// 确保所有线程都把数据加载完了
        // --- 步骤 B: 在 Shared Mem 中计算部分积 ---
        for (int k = 0; k < TILE_WIDTH; ++k) {
            pValue += ds_A[ty][k] * ds_B[k][tx];
        }
        __syncthreads();// 否则有的线程还在算，有的线程已经把 Shared Mem 覆盖了
    }
    if (row < N && col < N) {
        C[row * N + col] = pValue;
    }
}
int main() {
    int N = 1024;
    size_t size = N * N * sizeof(float);
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size); cudaMalloc(&d_B, size); cudaMalloc(&d_C, size);
    // 配置 Kernel
    dim3 threadsPerBlock(TILE_WIDTH, TILE_WIDTH);
    dim3 blocksPerGrid((N + TILE_WIDTH - 1) / TILE_WIDTH,
                       (N + TILE_WIDTH - 1) / TILE_WIDTH);
    matrixMulTiled<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    std::cout << "Tiled MatMul Done!" << std::endl;
    return 0;
}
```